

Hotel Business Operation Analysis

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
```

```
file_path='/content/hotel_bookings.csv'
```

```
df=pd.read_csv(file_path)
```

```
df.sample(5)
```

	hotel	is_canceled	lead_time	arrival_date_year	arrival_date_month	arrival_date_week_number
118678	City Hotel	0	172	2017	August	34
100020	City Hotel	0	19	2016	October	43
50871	City Hotel	1	68	2016	May	20
102233	City Hotel	0	212	2016	November	47
69910	City Hotel	1	181	2017	June	23

5 rows × 32 columns

```
df.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 119390 entries, 0 to 119389
Data columns (total 32 columns):
#   Column                                Non-Null Count  Dtype
---  -
0   hotel                                119390 non-null object
1   is_canceled                          119390 non-null int64
2   lead_time                           119390 non-null int64
3   arrival_date_year                    119390 non-null int64
4   arrival_date_month                   119390 non-null object
5   arrival_date_week_number             119390 non-null int64
6   arrival_date_day_of_month            119390 non-null int64
7   stays_in_weekend_nights              119390 non-null int64
8   stays_in_week_nights                 119390 non-null int64
9   adults                               119390 non-null int64
10  children                             119386 non-null float64
11  babies                               119390 non-null int64
12  meal                                 119390 non-null object
13  country                              118902 non-null object
14  market_segment                       119390 non-null object
15  distribution_channel                  119390 non-null object
16  is_repeated_guest                     119390 non-null int64
17  previous_cancellations                 119390 non-null int64
18  previous_bookings_not_canceled        119390 non-null int64
19  reserved_room_type                    119390 non-null object
20  assigned_room_type                    119390 non-null object
21  booking_changes                       119390 non-null int64
22  deposit_type                          119390 non-null object
23  agent                                103050 non-null float64
24  company                               6797 non-null float64
25  days_in_waiting_list                  119390 non-null int64
26  customer_type                         119390 non-null object
27  adr                                   119390 non-null float64
28  required_car_parking_spaces           119390 non-null int64
```

```

29 total_of_special_requests    119390 non-null    int64
30 reservation_status          119390 non-null    object
31 reservation_status_date      119390 non-null    object
dtypes: float64(4), int64(16), object(12)
memory usage: 29.1+ MB

```

```
df.describe()
```

	is_canceled	lead_time	arrival_date_year	arrival_date_week_number	arrival_date_day_of_mon
count	119390.000000	119390.000000	119390.000000	119390.000000	119390.000000
mean	0.370416	104.011416	2016.156554	27.165173	15.7982
std	0.482918	106.863097	0.707476	13.605138	8.7808
min	0.000000	0.000000	2015.000000	1.000000	1.0000
25%	0.000000	18.000000	2016.000000	16.000000	8.0000
50%	0.000000	69.000000	2016.000000	28.000000	16.0000
75%	1.000000	160.000000	2017.000000	38.000000	23.0000
max	1.000000	737.000000	2017.000000	53.000000	31.0000

▼ Data PreProcessing

```
print(df.isnull().sum())
```

```

hotel                                0
is_canceled                          0
lead_time                            0
arrival_date_year                     0
arrival_date_month                    0
arrival_date_week_number              0
arrival_date_day_of_month             0
stays_in_weekend_nights               0
stays_in_week_nights                 0
adults                               0
children                             4
babies                               0
meal                                  0
country                             488
market_segment                       0
distribution_channel                  0
is_repeated_guest                    0
previous_cancellations                0
previous_bookings_not_canceled        0
reserved_room_type                   0
assigned_room_type                   0
booking_changes                       0
deposit_type                         0
agent                               16340
company                             112593
days_in_waiting_list                 0
customer_type                         0
adr                                  0
required_car_parking_spaces           0
total_of_special_requests             0
reservation_status                    0
reservation_status_date               0
dtype: int64

```

```

# Fill missing values in 'children' with the median
median_children = df['children'].median()
df['children'].fillna(median_children, inplace=True)

# Fill missing values in 'country' with the mode
mode_country = df['country'].mode()[0]
df['country'].fillna(mode_country, inplace=True)

```

```
# Fill missing values in 'agent' and 'company' with mode as these fields have most repetitive Agent and Company
```

```

# Fill missing values in 'agent' and 'company' with mode as these fields have mode repetitive Agent and
mode_agent = df['agent'].mode()[0]
df['agent'].fillna(mode_agent, inplace=True)

mode_company = df['company'].mode()[0]
df['company'].fillna(mode_company, inplace=True)

```

/tmp/ipykernel_5126/712723338.py:3: FutureWarning: A value is trying to be set on a copy of a DataFrame
The behavior will change in pandas 3.0. This inplace method will never work because the intermediate obj

For example, when doing 'df[col].method(value, inplace=True)', try using 'df.method({col: value}, inplac

```

df['children'].fillna(median_children, inplace=True)
/tmp/ipykernel_5126/712723338.py:7: FutureWarning: A value is trying to be set on a copy of a DataFrame
The behavior will change in pandas 3.0. This inplace method will never work because the intermediate obj

```

For example, when doing 'df[col].method(value, inplace=True)', try using 'df.method({col: value}, inplac

```

df['country'].fillna(mode_country, inplace=True)
/tmp/ipykernel_5126/712723338.py:11: FutureWarning: A value is trying to be set on a copy of a DataFrame
The behavior will change in pandas 3.0. This inplace method will never work because the intermediate obj

```

For example, when doing 'df[col].method(value, inplace=True)', try using 'df.method({col: value}, inplac

```

df['agent'].fillna(mode_agent, inplace=True)
/tmp/ipykernel_5126/712723338.py:14: FutureWarning: A value is trying to be set on a copy of a DataFrame
The behavior will change in pandas 3.0. This inplace method will never work because the intermediate obj

```

For example, when doing 'df[col].method(value, inplace=True)', try using 'df.method({col: value}, inplac

```
df['company'].fillna(mode_company, inplace=True)
```

```
print('\nMissing values after imputation:\n', df.isnull().sum())
```

```

Missing values after imputation:
hotel                                0
is_canceled                          0
lead_time                            0
arrival_date_year                     0
arrival_date_month                    0
arrival_date_week_number              0
arrival_date_day_of_month             0
stays_in_weekend_nights               0
stays_in_week_nights                 0
adults                               0
children                             0
babies                               0
meal                                  0
country                              0
market_segment                       0
distribution_channel                 0
is_repeated_guest                    0
previous_cancellations               0
previous_bookings_not_canceled       0
reserved_room_type                   0
assigned_room_type                   0
booking_changes                      0
deposit_type                         0
agent                                0
company                              0
days_in_waiting_list                0
customer_type                        0
adr                                  0
required_car_parking_spaces          0
total_of_special_requests             0
reservation_status                   0
reservation_status_date              0
arrival_date                         0
total_stay_nights                    0
total_revenue                        0
dtype: int64

```

```
df.sample(5)
```

adr	required_car_parking_spaces	total_of_special_requests	reservation_status	reservation_status_date
55.8	0	2	Canceled	2016-10-01
91.0	0	0	Check-Out	2017-04-18
85.0	0	0	Check-Out	2016-05-05
99.0	0	0	Canceled	2015-11-23
90.0	0	0	Canceled	2015-11-06

```
# Combine year, month, and day to create a proper 'arrival_date' column
df['arrival_date'] = pd.to_datetime(df['arrival_date_year'].astype(str) + '-' +
                                   df['arrival_date_month'].astype(str) + '-' +
                                   df['arrival_date_day_of_month'].astype(str), errors='coerce')
```

```
# Convert 'reservation_status_date' to datetime
df['reservation_status_date'] = pd.to_datetime(df['reservation_status_date'], errors='coerce')
```

```
print('Data types after date conversion:')
df[['arrival_date', 'reservation_status_date']].info()
```

```
Data types after date conversion:
<class 'pandas.core.frame.DataFrame'>
Index: 87376 entries, 0 to 119389
Data columns (total 2 columns):
#   Column                Non-Null Count  Dtype
---  -
0   arrival_date           87376 non-null  datetime64[ns]
1   reservation_status_date 87376 non-null  datetime64[ns]
dtypes: datetime64[ns](2)
memory usage: 2.0 MB
```

✓ Handling Duplicate Entries

```
print(f"Initial number of rows: {df.shape[0]}")
duplicate_rows_count = df.duplicated().sum()
print(f"Number of duplicate rows found: {duplicate_rows_count}")

if duplicate_rows_count > 0:
    df.drop_duplicates(inplace=True)
    print(f"Number of rows after removing duplicates: {df.shape[0]}")
else:
    print("No duplicate rows found.")
```

```
Initial number of rows: 119390
Number of duplicate rows found: 32014
Number of rows after removing duplicates: 87376
```

```
df.info()
```

```
<class 'pandas.core.frame.DataFrame'>
Index: 87376 entries, 0 to 119389
Data columns (total 33 columns):
#   Column                Non-Null Count  Dtype
---  -
0   hotel                  87376 non-null  object
1   is_canceled            87376 non-null  int64
```

```

2  lead_time                87376 non-null int64
3  arrival_date_year        87376 non-null int64
4  arrival_date_month       87376 non-null object
5  arrival_date_week_number 87376 non-null int64
6  arrival_date_day_of_month 87376 non-null int64
7  stays_in_weekend_nights  87376 non-null int64
8  stays_in_week_nights     87376 non-null int64
9  adults                   87376 non-null int64
10 children                 87376 non-null float64
11 babies                   87376 non-null int64
12 meal                     87376 non-null object
13 country                  87376 non-null object
14 market_segment           87376 non-null object
15 distribution_channel      87376 non-null object
16 is_repeated_guest         87376 non-null int64
17 previous_cancellations    87376 non-null int64
18 previous_bookings_not_canceled 87376 non-null int64
19 reserved_room_type        87376 non-null object
20 assigned_room_type        87376 non-null object
21 booking_changes           87376 non-null int64
22 deposit_type              87376 non-null object
23 agent                     87376 non-null float64
24 company                   87376 non-null float64
25 days_in_waiting_list      87376 non-null int64
26 customer_type             87376 non-null object
27 adr                       87376 non-null float64
28 required_car_parking_spaces 87376 non-null int64
29 total_of_special_requests 87376 non-null int64
30 reservation_status        87376 non-null object
31 reservation_status_date    87376 non-null datetime64[ns]
32 arrival_date              87376 non-null datetime64[ns]
dtypes: datetime64[ns](2), float64(4), int64(16), object(11)
memory usage: 22.7+ MB

```

EDA :Lets Deep Dive into Data Insights for informed Decision making

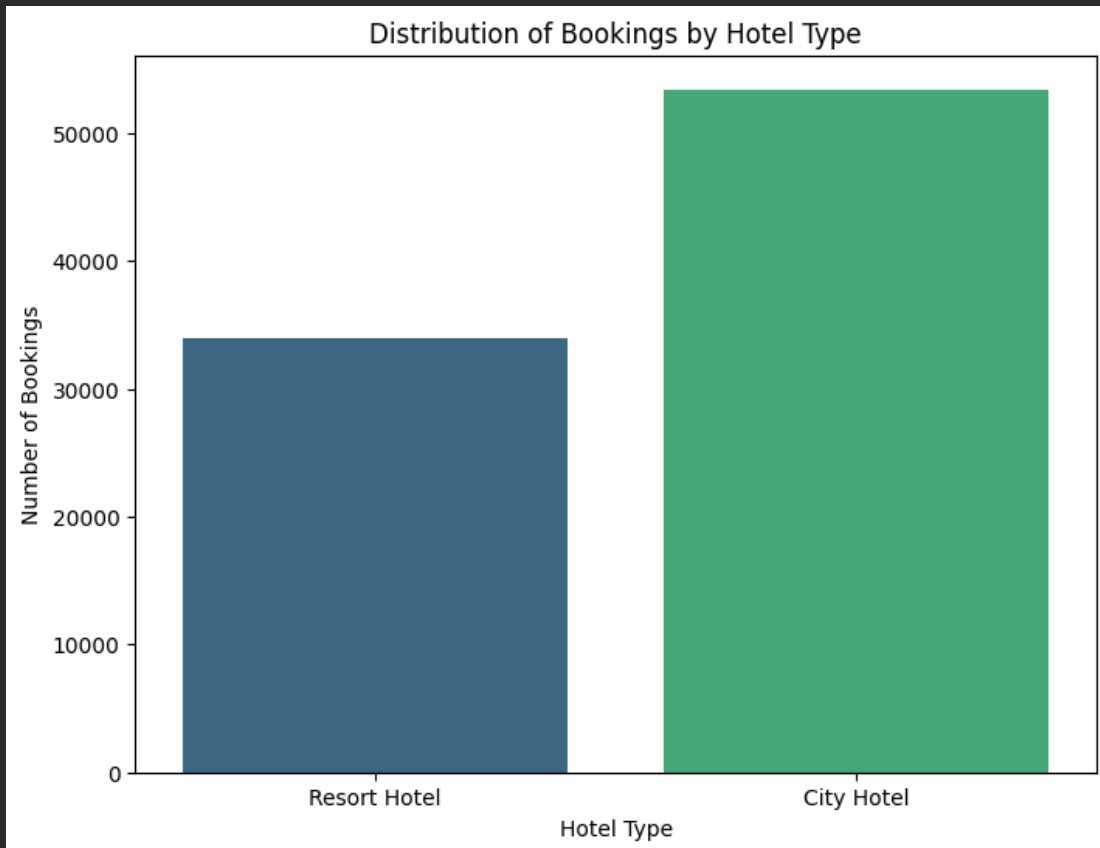
✓ Analysis of Booking Distribution

✓ Distribution of Bookings by Hotel Type

```

plt.figure(figsize=(8, 6))
sns.countplot(data=df, x='hotel', palette='viridis', hue='hotel', legend=False)
plt.title('Distribution of Bookings by Hotel Type')
plt.xlabel('Hotel Type')
plt.ylabel('Number of Bookings')
plt.show()

```

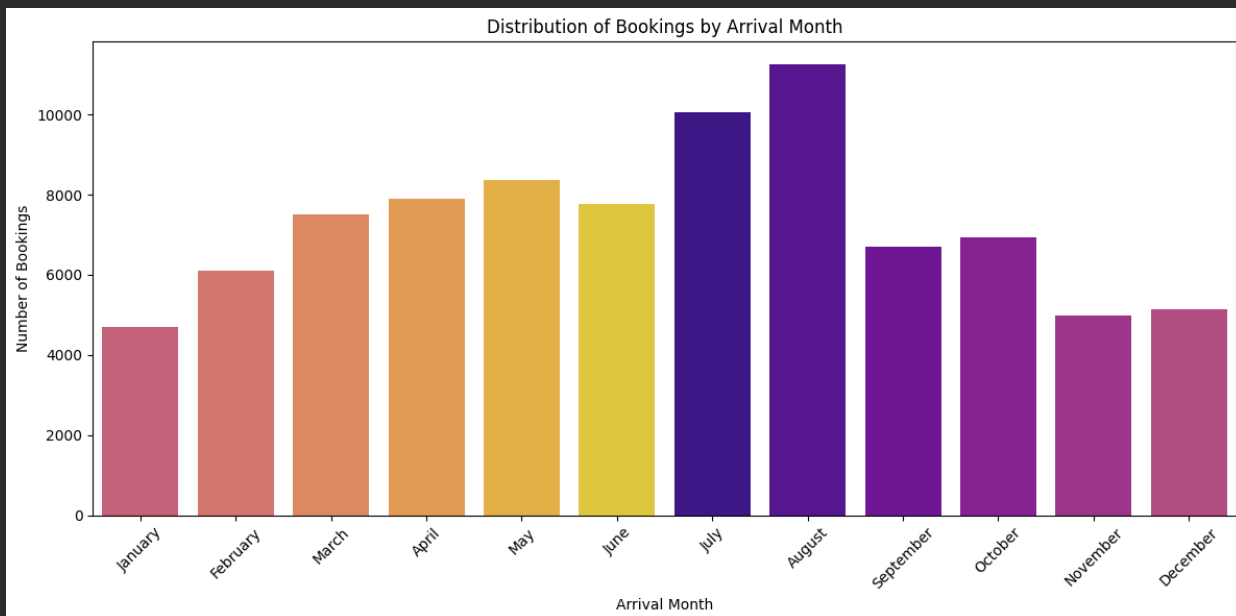


✓ Business Impact:

- Helps in targeting marketing efforts appropriately.
- Shows the popularity of Hotel Type

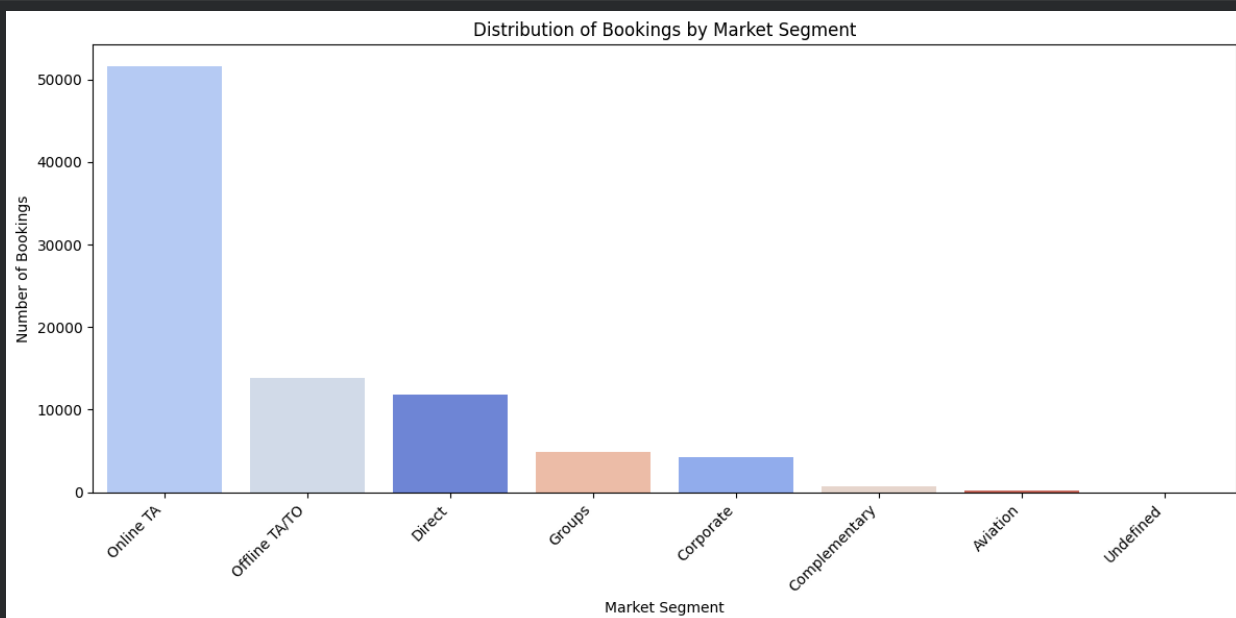
✓ Distribution of Bookings by Arrival Month

```
plt.figure(figsize=(12, 6))
sns.countplot(data=df, x='arrival_date_month', order=['January', 'February', 'March', 'April', 'May', 'June', 'July', 'August', 'September', 'October', 'November', 'December'])
plt.title('Distribution of Bookings by Arrival Month')
plt.xlabel('Arrival Month')
plt.ylabel('Number of Bookings')
plt.xticks(rotation=45)
plt.tight_layout()
plt.show()
```



✓ Distribution of Bookings by Market Segment

```
plt.figure(figsize=(12, 6))
sns.countplot(data=df, x='market_segment', palette='coolwarm', order=df['market_segment'].value_counts())
plt.title('Distribution of Bookings by Market Segment')
plt.xlabel('Market Segment')
plt.ylabel('Number of Bookings')
plt.xticks(rotation=45, ha='right')
plt.tight_layout()
plt.show()
```



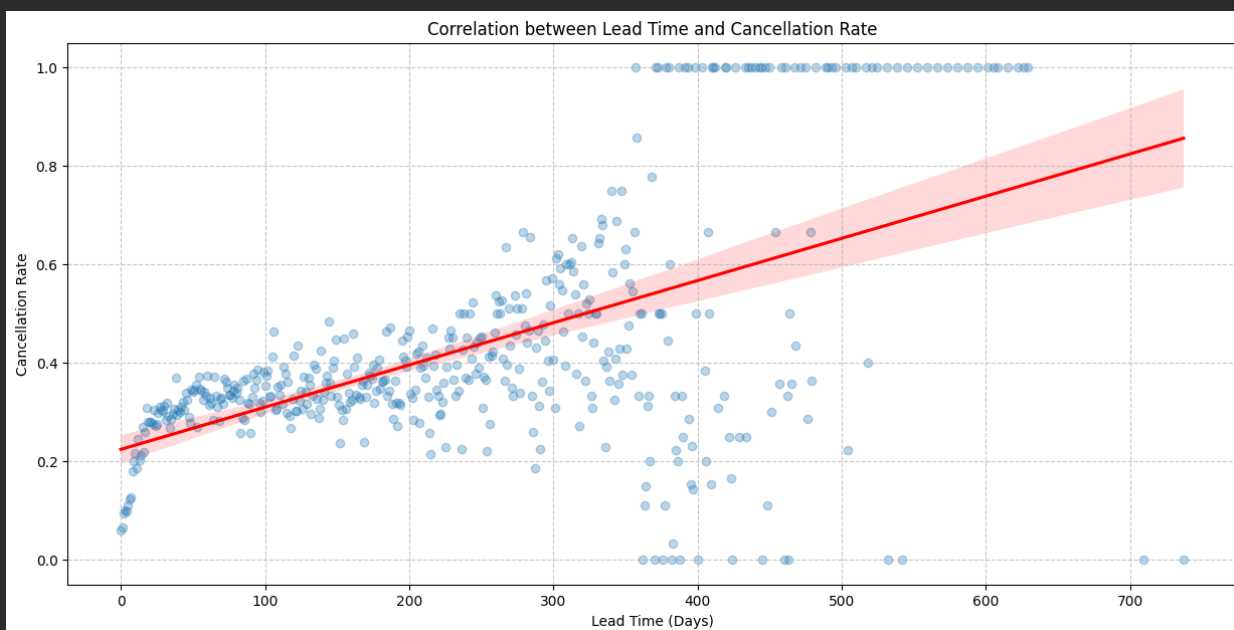
✓ Correlation between Lead Time and Cancellation Rate

```

cancellation_by_lead_time = df.groupby('lead_time')['is_canceled'].mean().reset_index()

plt.figure(figsize=(15, 7))
sns.regplot(data=cancellation_by_lead_time, x='lead_time', y='is_canceled', scatter_kws={'alpha':0.3},
plt.title('Correlation between Lead Time and Cancellation Rate')
plt.xlabel('Lead Time (Days)')
plt.ylabel('Cancellation Rate')
plt.grid(True, linestyle='--', alpha=0.7)
plt.show()

```



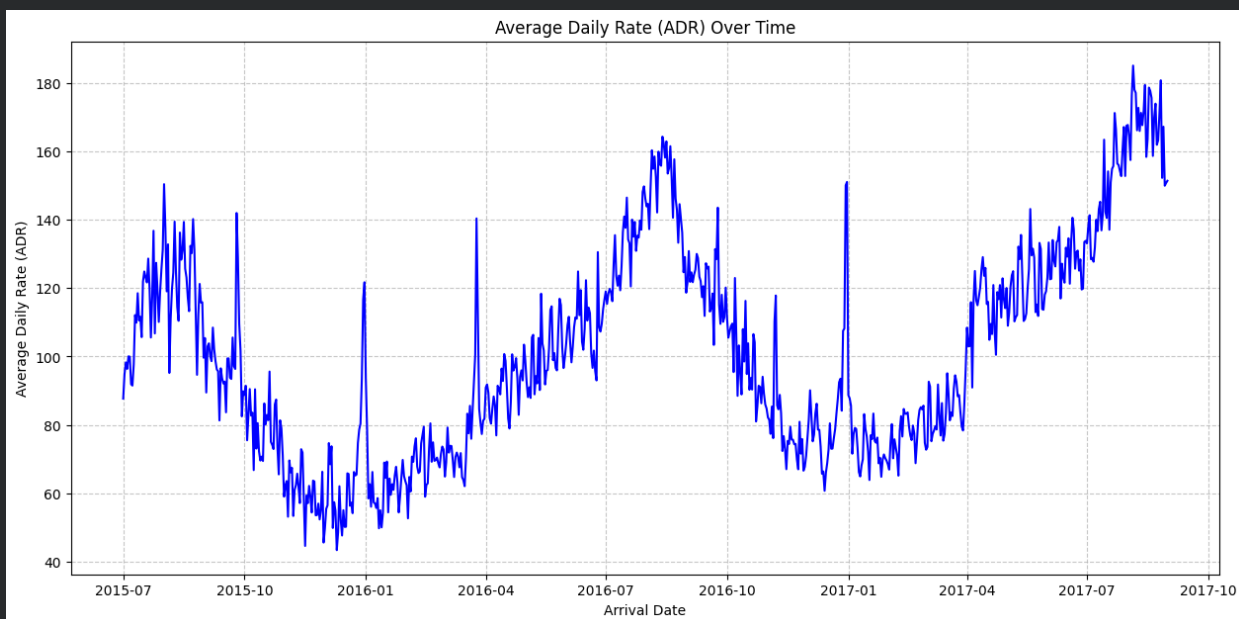
✓ Trend of Average Daily Rate (ADR) Over Time

```

adr_over_time = df.groupby('arrival_date')['adr'].mean().reset_index()

plt.figure(figsize=(15, 7))
sns.lineplot(data=adr_over_time, x='arrival_date', y='adr', color='blue')
plt.title('Average Daily Rate (ADR) Over Time')
plt.xlabel('Arrival Date')
plt.ylabel('Average Daily Rate (ADR)')
plt.grid(True, linestyle='--', alpha=0.7)
plt.show()

```



Double-click (or enter) to edit

✓ Impact of Special Requests on Cancellation Rate (Guest Satisfaction)

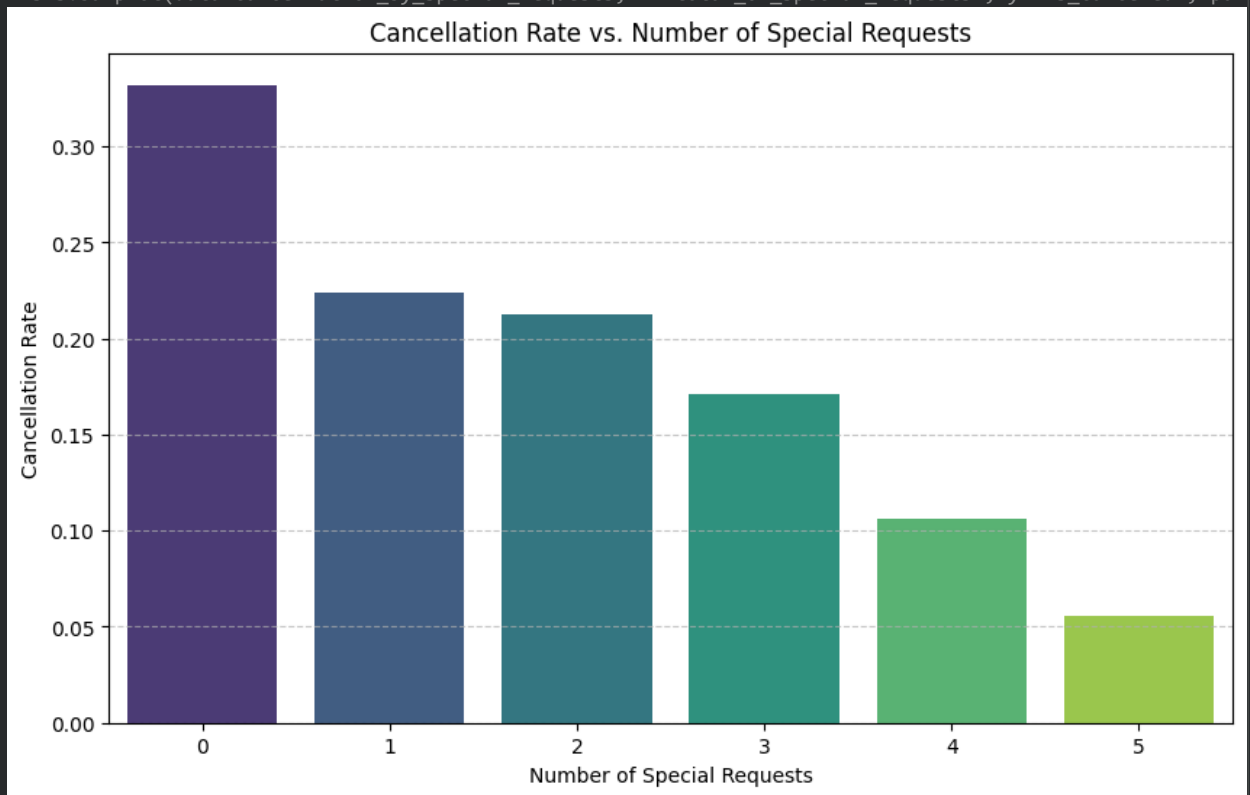
```
cancellation_by_special_requests = df.groupby('total_of_special_requests')['is_canceled'].mean().reset_index()

plt.figure(figsize=(10, 6))
sns.barplot(data=cancellation_by_special_requests, x='total_of_special_requests', y='is_canceled', palette='magma')
plt.title('Cancellation Rate vs. Number of Special Requests')
plt.xlabel('Number of Special Requests')
plt.ylabel('Cancellation Rate')
plt.xticks(rotation=0)
plt.grid(axis='y', linestyle='--', alpha=0.7)
plt.show()
```

/tmp/ipykernel_5126/1788076099.py:4: FutureWarning:

Passing `palette` without assigning `hue` is deprecated and will be removed in v0.14.0. Assign the `x` variable to `hue` to preserve the current behavior.

```
sns.barplot(data=cancellation_by_special_requests, x='total_of_special_requests', y='is_canceled', palette='magma')
```



```
average_lead_time = df['lead_time'].mean()
print(f"The average lead time for bookings is: {average_lead_time:.2f} days")
```

The average lead time for bookings is: 79.91 days

✓ Business Impact:

- Avg Lead Time helps in predicting booking trends and planning marketing campaigns accordingly.

```
canceled_bookings = df['is_canceled'].sum()
print(f"Total number of canceled bookings: {canceled_bookings}")
```

Total number of canceled bookings: 24024

Business Impact:

- Helps in developing strategies to reduce cancellations.
- Provides insight into booking reliability.

```
most_common_arrival_month = df['arrival_date_month'].mode()[0]
print(f"The most common arrival month for bookings is: {most_common_arrival_month}")
```

The most common arrival month for bookings is: August

Business Impact:

- Aids in resource and staff planning for peak seasons.
- Identifies peak booking periods.

```
average_special_requests = df['total_of_special_requests'].mean()
print(f"The average number of special requests per booking is: {average_special_requests:.2f}")
```

The average number of special requests per booking is: 0.70

Business Impact:

- Improves customer satisfaction by meeting their requests.
- Understands customer service requirements.

```
most_bookings_country = df['country'].value_counts().idxmax()
print(f"The country with the highest number of bookings is: {most_bookings_country}")
```

The country with the highest number of bookings is: PRT

Business Impact:

- Focuses marketing efforts on key regions.
- Identifies major customer demographics.

```
avg_adr_by_hotel = df.groupby('hotel')['adr'].mean().reset_index()
print("Average Daily Rate (ADR) for each Hotel Type:")
print(avg_adr_by_hotel)
```

Average Daily Rate (ADR) for each Hotel Type:

	hotel	adr
0	City Hotel	110.985832
1	Resort Hotel	99.041495

Business Impact:

- Informs pricing and revenue management strategies.
- Compares financial performance of different hotel types.

```
car_parking_required = df[df['required_car_parking_spaces'] > 0].shape[0]
total_bookings = df.shape[0]
percentage_car_parking = (car_parking_required / total_bookings) * 100
print(f"Percentage of guests who required car parking spaces: {percentage_car_parking:.2f}%")
```

Percentage of guests who required car parking spaces: 8.37%

Business Impact:

- Ensures adequate parking availability for guests.
- Assesses the demand for parking facilities.

```
average_week_nights = df['stays_in_week_nights'].mean()
average_weekend_nights = df['stays_in_weekend_nights'].mean()

print(f"Average stay duration in week nights: {average_week_nights:.2f} nights")
print(f"Average stay duration in weekend nights: {average_weekend_nights:.2f} nights")
```

```
Average stay duration in week nights: 2.63 nights
Average stay duration in weekend nights: 1.01 nights
```

Business Impact:

- Helps in optimizing room occupancy and pricing strategies.
- Helps in Understanding Booking Pattern

```
bookings_through_agent = df['agent'].count()
print(f"Number of bookings made through travel agents: {bookings_through_agent}")
```

```
Number of bookings made through travel agents: 87376
```

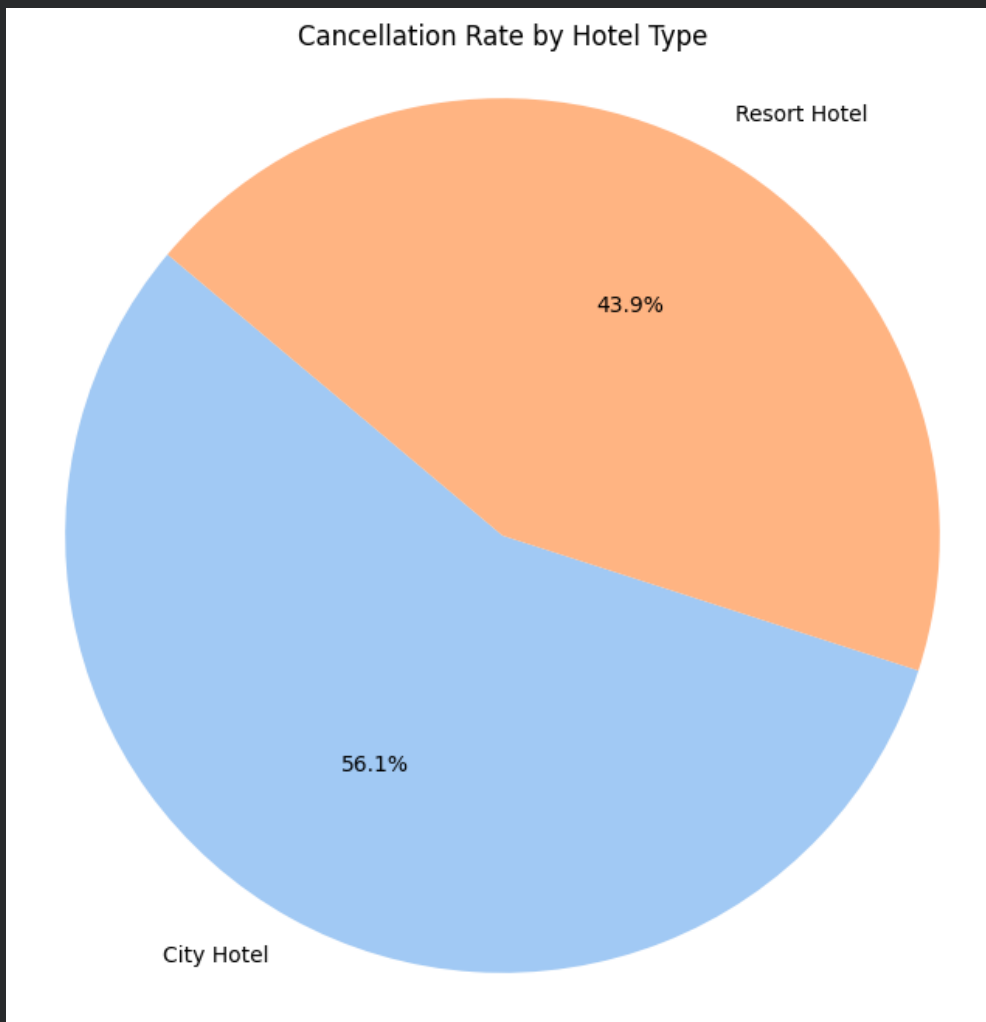
Business Impact:

- Helps in managing relationships with travel agents effectively.
- Identifies the importance of travel agents in booking processes.

```
cancellation_rate_by_hotel = df.groupby('hotel')['is_canceled'].mean().reset_index()
print("Cancellation Rate for each Hotel Type:")
print(cancellation_rate_by_hotel)
```

```
Cancellation Rate for each Hotel Type:
   hotel  is_canceled
0  City Hotel    0.300391
1  Resort Hotel    0.234911
```

```
plt.figure(figsize=(8, 8))
plt.pie(cancellation_rate_by_hotel['is_canceled'], labels=cancellation_rate_by_hotel['hotel'], autopct=
plt.title('Cancellation Rate by Hotel Type')
plt.axis('equal') # Equal aspect ratio ensures that pie is drawn as a circle.
plt.show()
```



Business Impact:

- Helps in developing strategies to reduce cancellations for specific hotel types.
- Helps in Identifying which hotel type has a higher cancellation rate.

Start coding or [generate](#) with AI.

Distribution Channel with the Highest Number of Bookings

```
most_bookings_channel = df['distribution_channel'].value_counts().idxmax()
print(f"The distribution channel with the highest number of bookings is: {most_bookings_channel}")
```

The distribution channel with the highest number of bookings is: TA/T0

Business Impact:

- Helps in optimizing partnerships with various distribution channels.
- Identifies the most effective channels for acquiring bookings.
- Helps in focusing efforts on the most productive channels.
- Identifies the most effective booking channels.

Average Number of Previous Cancellations by Hotel Type

```
avg_prev_cancellations_by_hotel = df.groupby('hotel')['previous_cancellations'].mean().reset_index()
print("Average number of previous cancellations by hotel type:")
print(avg_prev_cancellations_by_hotel)
```

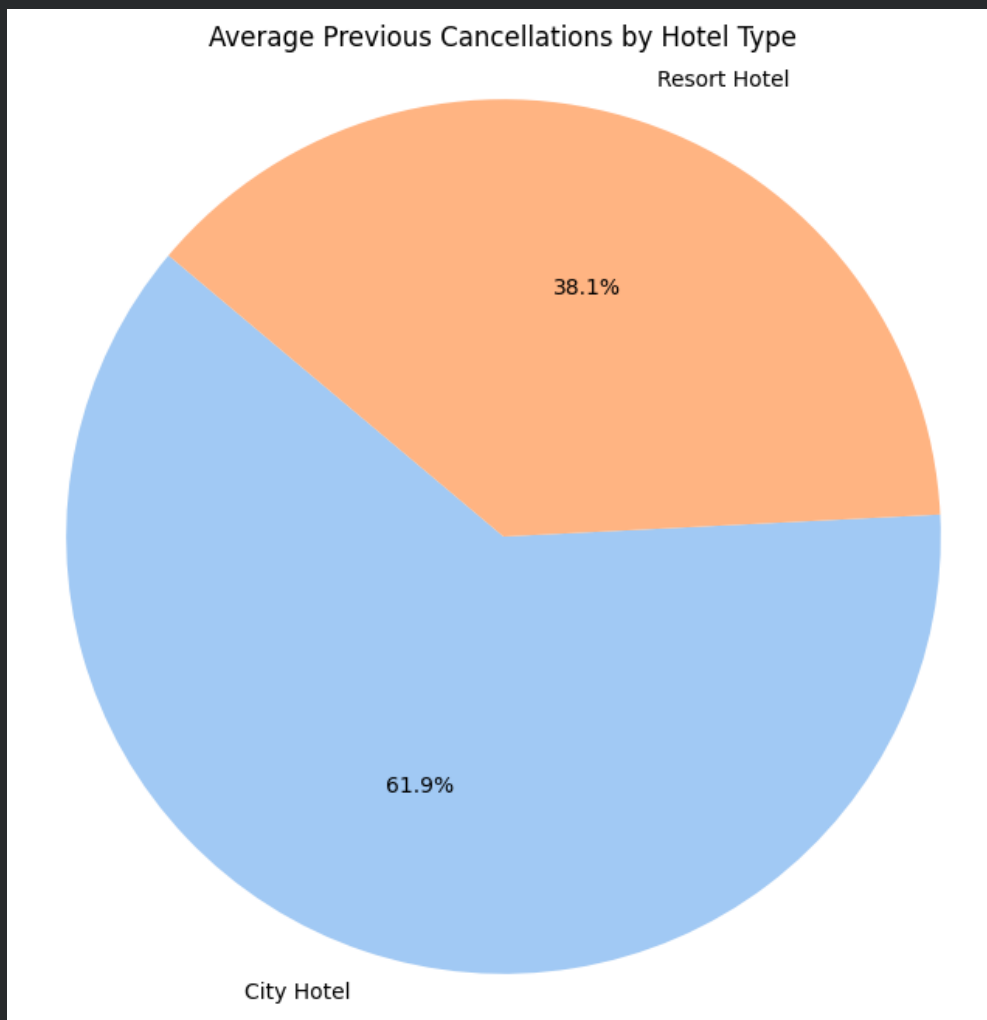
Average number of previous cancellations by hotel type:

	hotel	previous_cancellations
0	City Hotel	0.035768
1	Resort Hotel	0.022004

✓ Business Impact:

- Helps to understand customer loyalty and retention issues for each hotel type.
- Provides insights for targeted loyalty programs or service improvements to reduce cancellations.
- Helps in developing targeted strategies to reduce cancellations.
- Understands past cancellation behavior by hotel type.

```
plt.figure(figsize=(8, 8))
plt.pie(avg_prev_cancellations_by_hotel['previous_cancellations'], labels=avg_prev_cancellations_by_hotel['hotel'], autopct='%1.1f%%')
plt.title('Average Previous Cancellations by Hotel Type')
plt.axis('equal') # Equal aspect ratio ensures that pie is drawn as a circle.
plt.show()
```

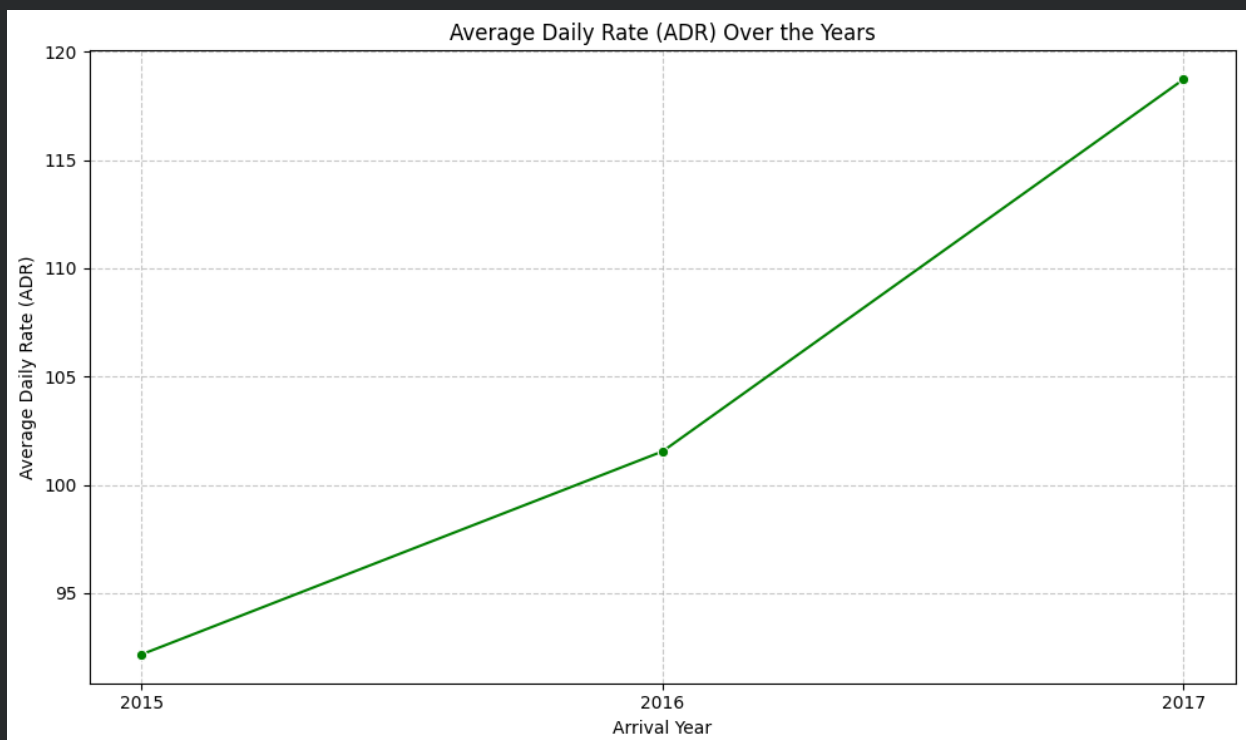


✓ Trend of Average Daily Rate (ADR) Over the Years

```
adr_by_year = df.groupby('arrival_date_year')['adr'].mean().reset_index()

plt.figure(figsize=(10, 6))
sns.lineplot(data=adr_by_year, x='arrival_date_year', y='adr', marker='o', color='green')
plt.title('Average Daily Rate (ADR) Over the Years')
plt.xlabel('Arrival Year')
```

```
plt.ylabel('Average Daily Rate (ADR)')
plt.grid(True, linestyle='--', alpha=0.7)
plt.xticks(adr_by_year['arrival_date_year'].unique()) # Ensure all years are shown as ticks
plt.tight_layout()
plt.show()
```



Business Impact:

- Provides insights into pricing strategies and their effectiveness over time.
- Helps in identifying periods of high and low demand, guiding future pricing adjustments.
- Informs long-term pricing and revenue management strategies.
- Identifies changes in ADR over time.

Month with the Highest Revenue

```
# Calculate total stay nights
df['total_stay_nights'] = df['stays_in_weekend_nights'] + df['stays_in_week_nights']

# Calculate total revenue per booking
df['total_revenue'] = df['adr'] * df['total_stay_nights']

# Group by month and sum the total revenue
monthly_revenue = df.groupby('arrival_date_month')['total_revenue'].sum().reset_index()

# Order months correctly for plotting
month_order = ['January', 'February', 'March', 'April', 'May', 'June', 'July', 'August', 'September', 'October']
monthly_revenue['arrival_date_month'] = pd.Categorical(monthly_revenue['arrival_date_month'], categories=month_order)
monthly_revenue = monthly_revenue.sort_values('arrival_date_month')

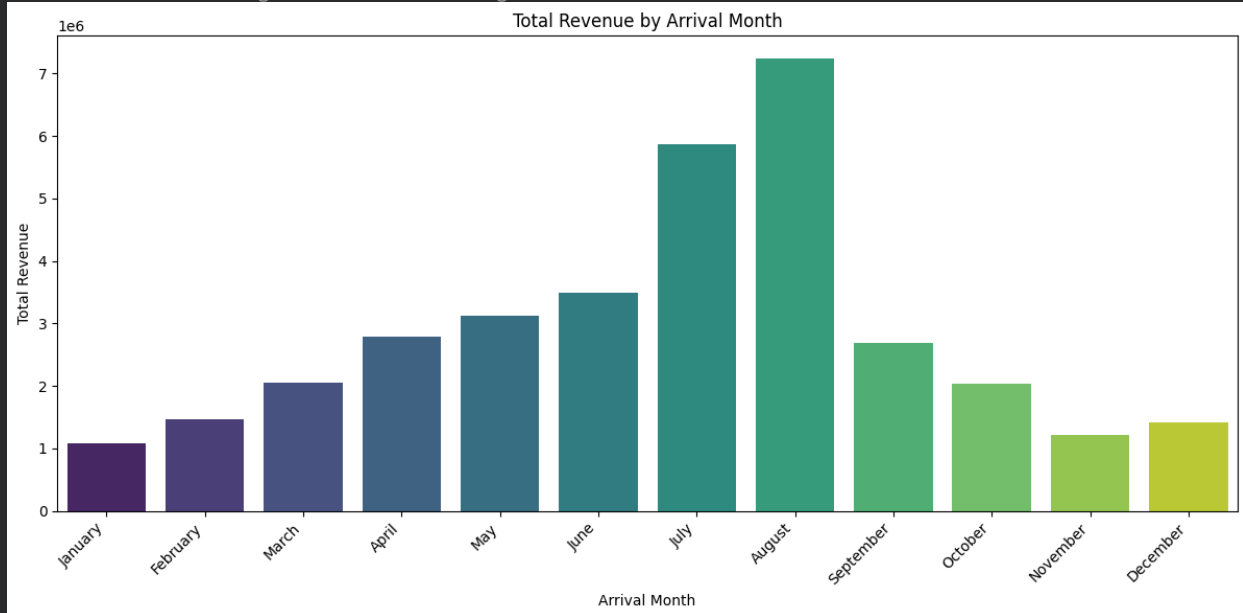
# Find the month with the highest revenue
highest_revenue_month = monthly_revenue.loc[monthly_revenue['total_revenue'].idxmax()]

print(f"The month with the highest revenue is: {highest_revenue_month['arrival_date_month']} with a total revenue of {highest_revenue_month['total_revenue']}")

# Visualize the monthly revenue
plt.figure(figsize=(12, 6))
sns.barplot(data=monthly_revenue, x='arrival_date_month', y='total_revenue', palette='viridis', hue='arrival_date_month')
plt.title('Total Revenue by Arrival Month')
```

```
plt.xlabel('Arrival Month')
plt.ylabel('Total Revenue')
plt.xticks(rotation=45, ha='right')
plt.tight_layout()
plt.show()
```

The month with the highest revenue is: August with a total revenue of 7240652.99



Business Impact:

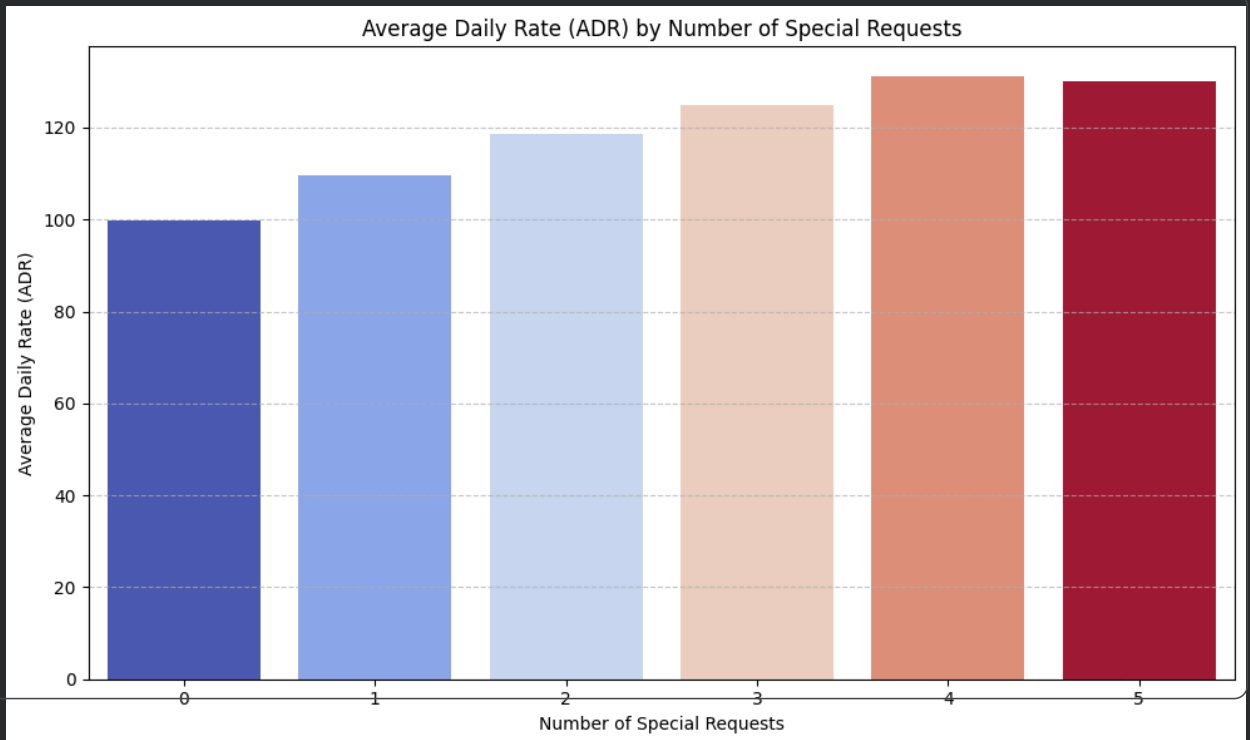
- Guides marketing efforts and promotions for months with lower revenue.
- Optimizes staffing and resource allocation based on revenue forecasts.
- Helps in strategic planning for peak revenue months.
- Helps in planning promotional activities and resource allocation.
- Identifies the most profitable months.

Impact of Special Requests on Average Daily Rate (ADR)

```
adr_by_special_requests = df.groupby('total_of_special_requests')['adr'].mean().reset_index()

plt.figure(figsize=(10, 6))
sns.barplot(data=adr_by_special_requests, x='total_of_special_requests', y='adr', palette='coolwarm', hue='total_of_special_requests')
plt.title('Average Daily Rate (ADR) by Number of Special Requests')
plt.xlabel('Number of Special Requests')
plt.ylabel('Average Daily Rate (ADR)')
plt.grid(axis='y', linestyle='--', alpha=0.7)
plt.xticks(rotation=0)
plt.tight_layout()
plt.show()

print("Average Daily Rate (ADR) by number of special requests:")
print(adr_by_special_requests)
```



Business Impact:

- Helps in understanding if guests with more special requests are associated with higher or lower ADR.
- Informs pricing strategies, potentially allowing for premium pricing for bookings with many special requests or offering incentives for fewer requests.
- Guides resource allocation for fulfilling special requests relative to the revenue generated.
- Helps in optimizing pricing strategies based on customer preferences.
- Understands how special requests influence pricing.

Average Stay Duration: Repeated Guests vs. New Guests

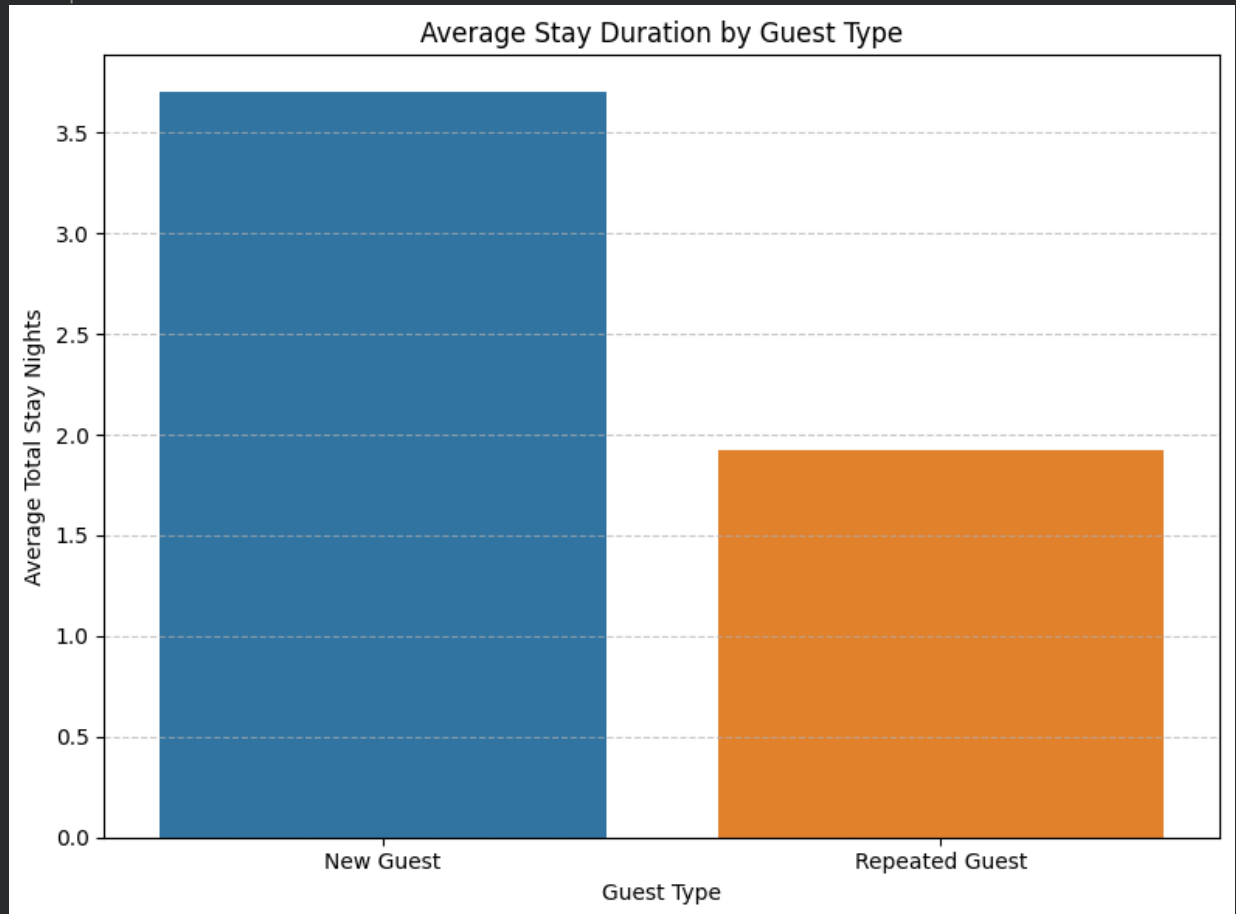
```
average_stay_duration_guests = df.groupby('is_repeated_guest')['total_stay_nights'].mean().reset_index()

# Map 0 and 1 to 'New Guest' and 'Repeated Guest' for better readability
average_stay_duration_guests['guest_type'] = average_stay_duration_guests['is_repeated_guest'].map({0:
print("Average stay duration for repeated vs. new guests:")
print(average_stay_duration_guests[['guest_type', 'total_stay_nights']])

plt.figure(figsize=(8, 6))
sns.barplot(data=average_stay_duration_guests, x='guest_type', y='total_stay_nights', palette='tab10',
plt.title('Average Stay Duration by Guest Type')
plt.xlabel('Guest Type')
plt.ylabel('Average Total Stay Nights')
plt.grid(axis='y', linestyle='--', alpha=0.7)
plt.tight_layout()
plt.show()
```


Average stay duration for repeated vs. new guests:

	guest_type	total_stay_nights
0	New Guest	3.700337
1	Repeated Guest	1.927086



✓ Business Impact:

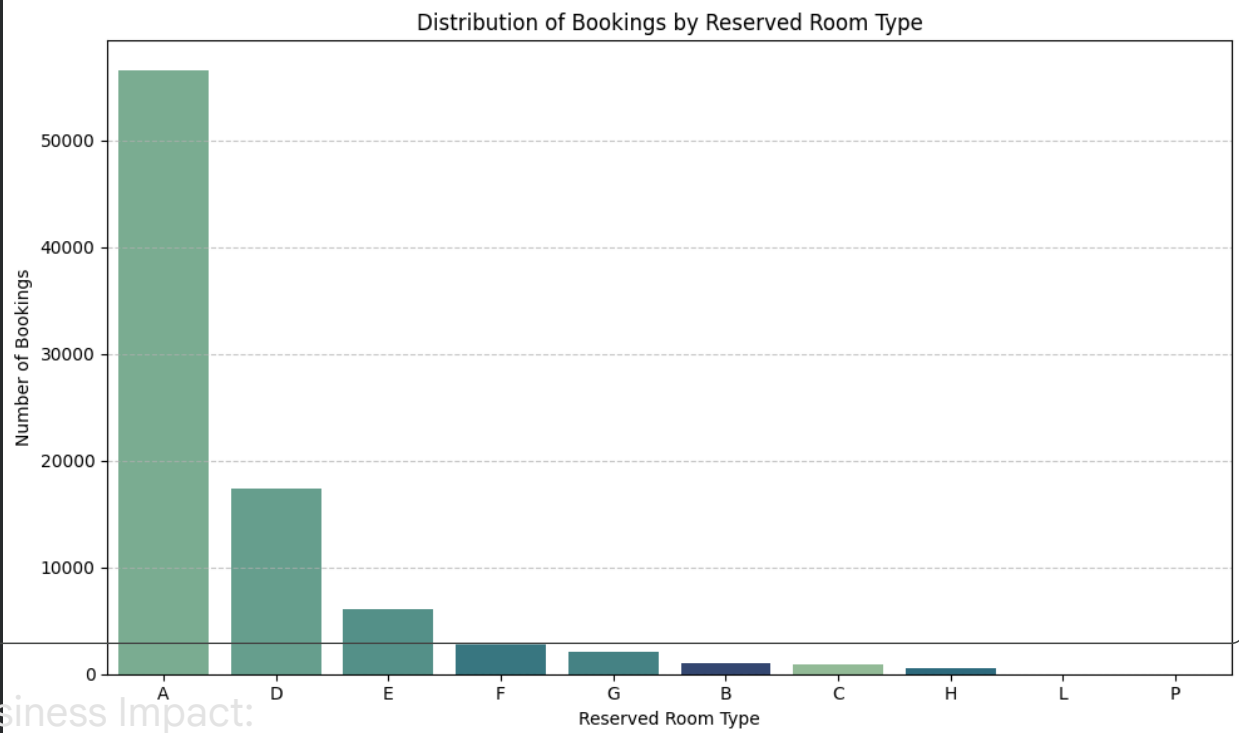
- Helps in tailoring loyalty programs and incentives to encourage longer stays from repeated guests.
- Provides insights for marketing efforts to attract new guests and convert them into repeated guests.
- Informs capacity planning and resource allocation based on the typical stay patterns of different guest segments.
- Helps in developing targeted loyalty programs and improving guest retention
- Understands guest behavior and loyalty.

✓ Room Type with the Highest Number of Bookings

```
most_booked_room_type = df['reserved_room_type'].value_counts().idxmax()
print(f"The room type with the highest number of bookings is: {most_booked_room_type}")

# Optional: Visualize the distribution of room types
plt.figure(figsize=(10, 6))
sns.countplot(data=df, x='reserved_room_type', order=df['reserved_room_type'].value_counts().index, palette='magma')
plt.title('Distribution of Bookings by Reserved Room Type')
plt.xlabel('Reserved Room Type')
plt.ylabel('Number of Bookings')
plt.grid(axis='y', linestyle='--', alpha=0.7)
plt.tight_layout()
plt.show()
```

The room type with the highest number of bookings is: A



Business Impact:

- Helps in optimizing inventory management and resource allocation for popular room types.
- Informs marketing and pricing strategies for different room types.
- Identifies customer preferences and demand for specific room configurations.
- Helps in optimizing room availability and marketing strategies.
- Identifies the most popular room types.

Factors Significantly Impacting Cancellation Rate (Logistic Regression Analysis)

```

from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler, OneHotEncoder
from sklearn.compose import ColumnTransformer
from sklearn.linear_model import LogisticRegression
from sklearn.pipeline import Pipeline

# Define target and features
X = df.drop('is_canceled', axis=1)
y = df['is_canceled']

# Identify numerical and categorical features
numerical_features = ['lead_time', 'stays_in_weekend_nights', 'stays_in_week_nights', 'adults', 'children',
                      'previous_cancellations', 'previous_bookings_not_canceled', 'booking_changes', 'a',
                      'required_car_parking_spaces', 'total_of_special_requests', 'total_stay_nights']
categorical_features = ['hotel', 'market_segment', 'distribution_channel', 'deposit_type', 'customer_type']

# Create a column transformer for preprocessing
preprocessor = ColumnTransformer(
    transformers=[
        ('num', StandardScaler(), numerical_features),
        ('cat', OneHotEncoder(handle_unknown='ignore'), categorical_features)
    ])

# Create a pipeline with preprocessing and logistic regression model
model = Pipeline(steps=[('preprocessor', preprocessor),
                        ('classifier', LogisticRegression(solver='liblinear', random_state=42))])

# Split the data into training and testing sets
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42, stratify=y)

# Train the model
model.fit(X_train, y_train)

print("Logistic Regression Model Trained Successfully!")

```

Logistic Regression Model Trained Successfully!

✓ Key Features influencing the Cancellation Rate using Logistic Regression Model

After training the model, let's look at the coefficients to understand which features have the most impact on the cancellation rate. Positive coefficients indicate that an increase in the feature value leads to a higher probability of cancellation, while negative coefficients indicate a lower probability.

```

# Get feature names after one-hot encoding
onehot_features = model.named_steps['preprocessor'].named_transformers_['cat'].get_feature_names_out(categorical_features)
feature_names = numerical_features + list(onehot_features)

# Get coefficients from the logistic regression model
coefficients = model.named_steps['classifier'].coef_[0]

# Create a DataFrame to store feature names and their coefficients
feature_importance = pd.DataFrame({'Feature': feature_names, 'Coefficient': coefficients})

# Sort by absolute coefficient value to see the most impactful features
feature_importance['Absolute_Coefficient'] = abs(feature_importance['Coefficient'])
feature_importance = feature_importance.sort_values(by='Absolute_Coefficient', ascending=False)

print("Top 15 Most Impactful Features on Cancellation Rate:")
display(feature_importance.head(15))

# Visualize the top N most impactful features
plt.figure(figsize=(12, 8))
sns.barplot(data=feature_importance.head(15), x='Coefficient', y='Feature', palette='coolwarm')
plt.title('Top 15 Feature Importance for Cancellation Prediction')
plt.xlabel('Coefficient Value')
plt.ylabel('Feature')
plt.tight_layout()
plt.show()

```

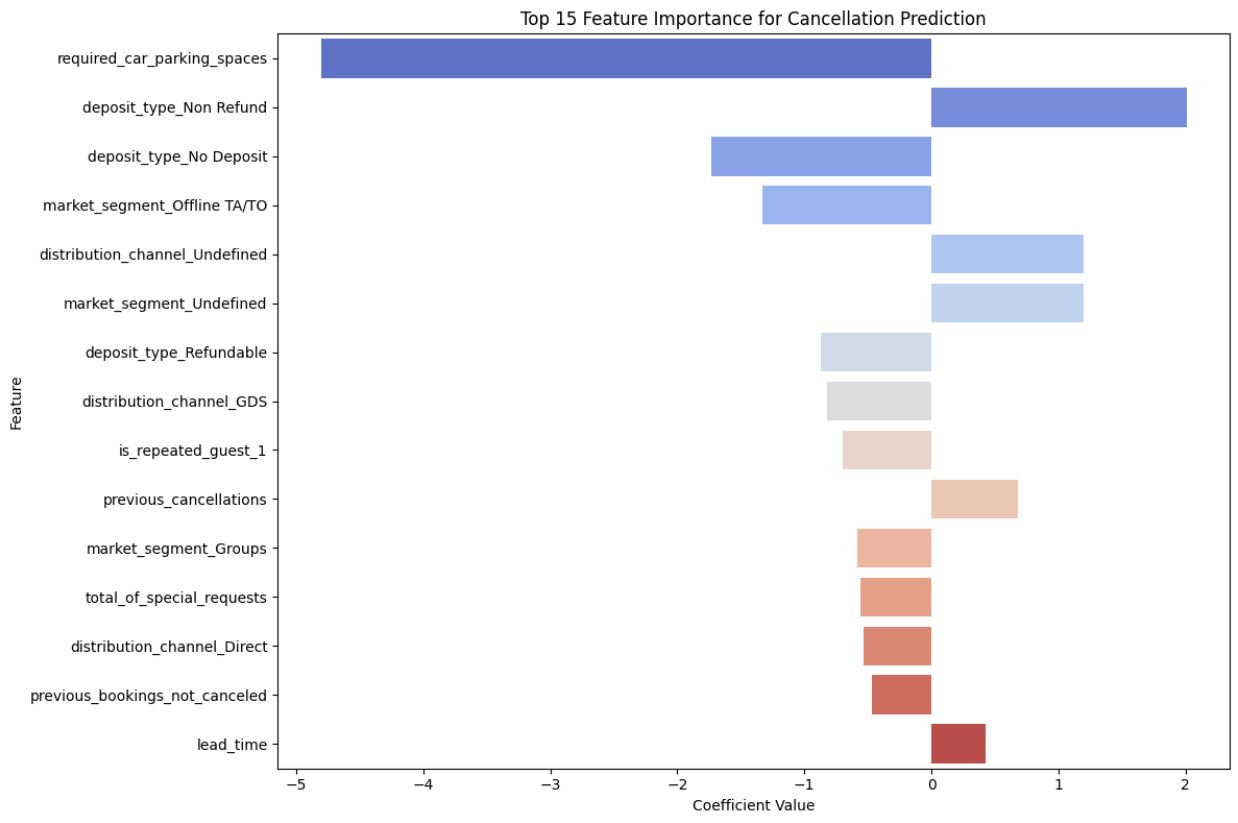
Top 15 Most Impactful Features on Cancellation Rate:

	Feature	Coefficient	Absolute_Coefficient
10	required_car_parking_spaces	-4.804728	4.804728
29	deposit_type_Non Refund	2.007698	2.007698
28	deposit_type_No Deposit	-1.731749	1.731749
20	market_segment_Offline TA/TO	-1.332978	1.332978
27	distribution_channel_Undefined	1.202722	1.202722
22	market_segment_Undefined	1.202681	1.202681
30	deposit_type_Refundable	-0.872124	0.872124
25	distribution_channel_GDS	-0.822074	0.822074
36	is_repeated_guest_1	-0.700092	0.700092
6	previous_cancellations	0.679502	0.679502
19	market_segment_Groups	-0.579382	0.579382
11	total_of_special_requests	-0.561323	0.561323
24	distribution_channel_Direct	-0.532715	0.532715
7	previous_bookings_not_canceled	-0.470802	0.470802
0	lead_time	0.423379	0.423379

/tmp/ipykernel_5126/2596402066.py:20: FutureWarning:

Passing `palette` without assigning `hue` is deprecated and will be removed in v0.14.0. Assign the `y` v

```
sns.barplot(data=feature_importance.head(15), x='Coefficient', y='Feature', palette='coolwarm')
```



Business Impact:

- **Informed Decision Making:** Identifies key drivers of cancellation, allowing hotels to proactively address issues and develop targeted strategies.
- **Risk Management:** Helps in assessing the risk of cancellation for new bookings based on their characteristics.
- **Strategic Interventions:** Guides interventions such as offering incentives to guests with high lead times or those with specific deposit types to reduce cancellation rates.
- **Resource Optimization:** Allows for better allocation of resources to manage bookings with a higher likelihood of cancellation.

✓ Variation of ADR with Number of Adults, Children, and Babies (Multiple Regression Analysis)

```
import statsmodels.formula.api as smf

# Define the regression formula
# ADR as the dependent variable, adults, children, and babies as independent variables
formula = 'adr ~ adults + children + babies'

# Fit the OLS (Ordinary Least Squares) model
model_adr = smf.ols(formula=formula, data=df).fit()

# Print the model summary to analyze coefficients and R-squared
print(model_adr.summary())
```

```

                        OLS Regression Results
=====
Dep. Variable:          adr      R-squared:                0.165
Model:                  OLS      Adj. R-squared:           0.165
Method:                 Least Squares      F-statistic:         5750.
Date:                   Wed, 27 May 2026      Prob (F-statistic):    0.00
Time:                   19:55:56      Log-Likelihood:       -4.6626e+05
No. Observations:      87376      AIC:                  9.325e+05
Df Residuals:           87372      BIC:                  9.326e+05
Df Model:                3
Covariance Type:        nonrobust
=====
                        coef      std err          t      P>|t|      [0.025      0.975]
-----
Intercept             61.2096      0.538     113.696      0.000      60.154      62.265
adults                 21.1645      0.272      77.937      0.000      20.632      21.697
children              38.6549      0.373     103.589      0.000      37.924      39.386
babies                 6.7059      1.497       4.479      0.000       3.771       9.641
=====
Omnibus:               165793.646      Durbin-Watson:           1.050
Prob(Omnibus):          0.000      Jarque-Bera (JB):       7354553136.777
Skew:                   13.383      Prob(JB):                0.00
Kurtosis:               1424.054      Cond. No.                19.4
=====

```

Notes:

[1] Standard Errors assume that the covariance matrix of the errors is correctly specified.

✓ Business Impact:

- **Dynamic Pricing Strategies:** The regression analysis provides quantifiable insights into how changes in the number of adults, children, and babies impact ADR. This allows hotels to implement more dynamic and precise pricing strategies, optimizing revenue based on booking demographics.
- **Targeted Promotions:** Understanding which demographic factors significantly influence ADR can help in designing targeted promotional campaigns. For example, if 'children' has a strong positive correlation with ADR, family packages can be priced accordingly.
- **Capacity and Resource Planning:** Insights into how different guest compositions affect ADR can aid in better planning of room types, amenities, and services. Hotels can optimize their offerings to cater to the most profitable guest segments.

- **Demand Forecasting:** By identifying the relationship between guest demographics and ADR, hotels can improve their demand forecasting models, leading to more accurate revenue projections and better inventory management.
- Helps in optimizing room rates based on guest demographics.
- Understands how different guest compositions affect pricing.

✓ Impact of Booking Changes on Special Requests (Guest Satisfaction)

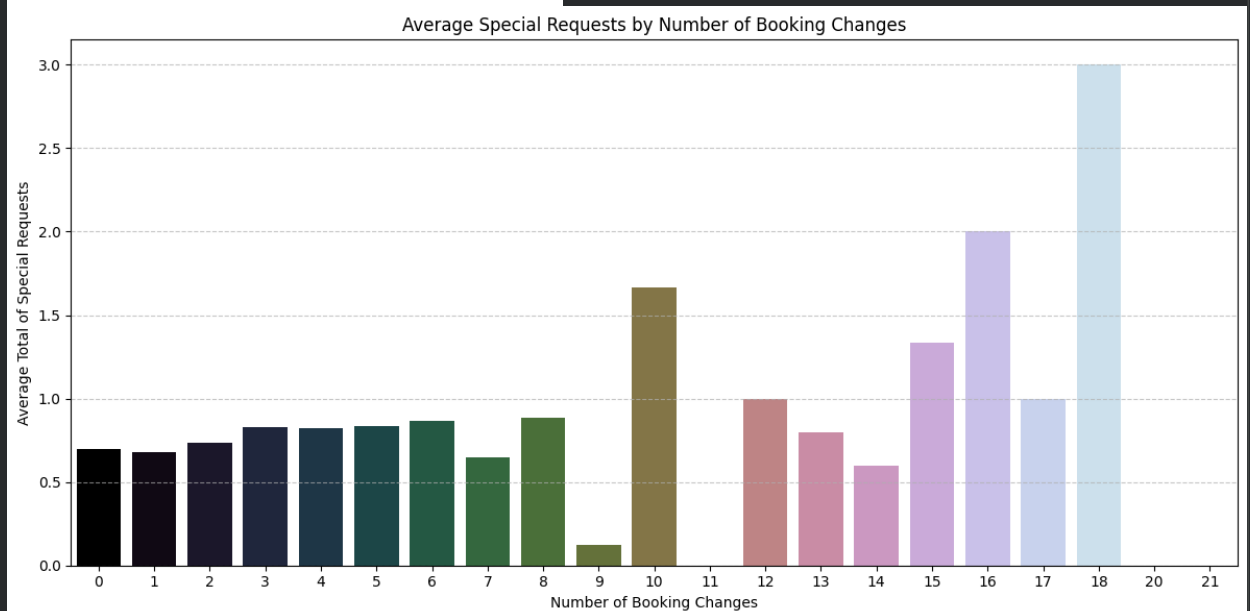
```
# Group by 'booking_changes' and calculate the mean of 'total_of_special_requests'
special_requests_by_changes = df.groupby('booking_changes')['total_of_special_requests'].mean().reset_index()

# Display the results
print("Average number of special requests by booking changes:")
display(special_requests_by_changes.head())

# Visualize the relationship
plt.figure(figsize=(12, 6))
sns.barplot(data=special_requests_by_changes, x='booking_changes', y='total_of_special_requests', palette='magma')
plt.title('Average Special Requests by Number of Booking Changes')
plt.xlabel('Number of Booking Changes')
plt.ylabel('Average Total of Special Requests')
plt.grid(axis='y', linestyle='--', alpha=0.7)
plt.tight_layout()
plt.show()
```

Average number of special requests by booking changes:

booking_changes	total_of_special_requests
0	0.696593
1	0.682048
2	0.737172
3	0.826286
4	0.820225



✓ Business Impact:

- **Customer Service Prioritization:** If there's a clear trend (e.g., more booking changes lead to fewer special requests), it suggests guests making changes might be less focused on personalization or are resigned to a simpler stay. This can help prioritize customer service efforts.

- **Operational Efficiency:** Understanding this relationship can help optimize the handling of special requests. If guests with many changes tend to have fewer special requests, staff can anticipate this and streamline their process.
- **Feedback Loop for Booking Process:** If a high number of booking changes consistently leads to a low number of special requests (or vice-versa), it might indicate issues or confusion in the initial booking process that needs to be addressed.
- **Targeted Guest Communication:** Hotels can tailor communications based on the number of booking changes. For guests with several changes, a simplified confirmation or a focus on essential information might be more appropriate, while for guests with no changes, encouraging special requests could enhance satisfaction.
- Helps in improving booking flexibility and customer satisfaction.
- Understands the relationship between booking changes and guest needs.

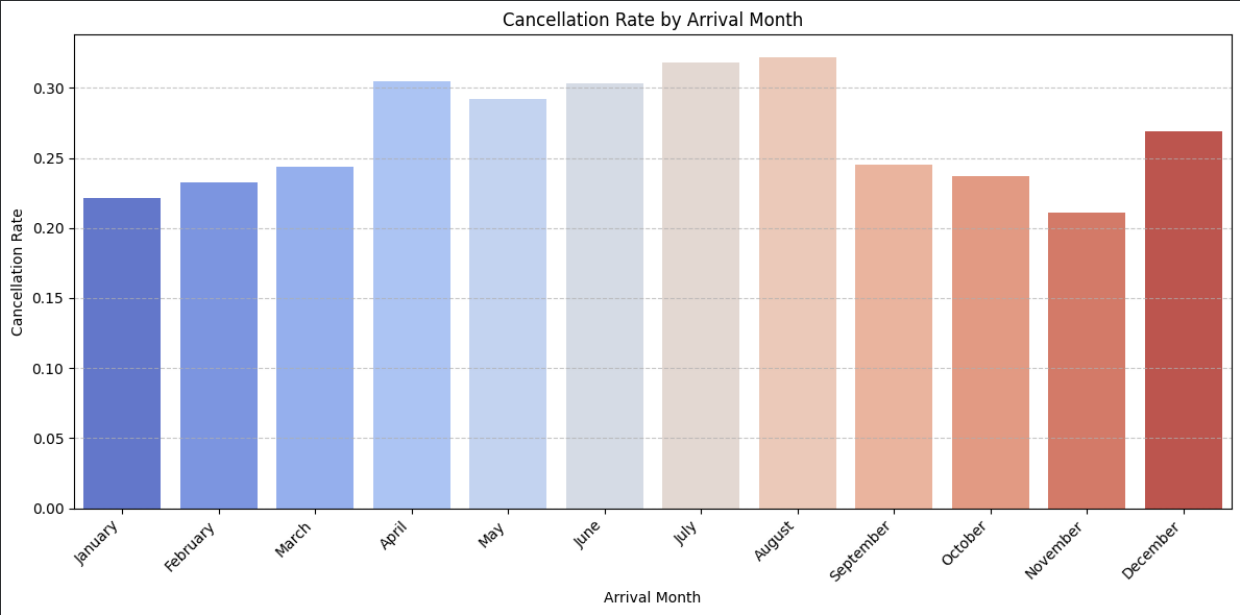
✓ Seasonal Impact on Booking Cancellations

```
# Calculate cancellation rate per month
cancellation_rate_by_month = df.groupby('arrival_date_month')['is_canceled'].mean().reset_index()

# Define the correct order of months
month_order = ['January', 'February', 'March', 'April', 'May', 'June', 'July', 'August', 'September', 'October', 'November', 'December']
cancellation_rate_by_month['arrival_date_month'] = pd.Categorical(cancellation_rate_by_month['arrival_date_month'], categories=month_order, ordered=True)
cancellation_rate_by_month = cancellation_rate_by_month.sort_values('arrival_date_month')

# Visualize the seasonal impact on cancellation rates
plt.figure(figsize=(12, 6))
sns.barplot(data=cancellation_rate_by_month, x='arrival_date_month', y='is_canceled', palette='coolwarm')
plt.title('Cancellation Rate by Arrival Month')
plt.xlabel('Arrival Month')
plt.ylabel('Cancellation Rate')
plt.xticks(rotation=45, ha='right')
plt.grid(axis='y', linestyle='--', alpha=0.7)
plt.tight_layout()
plt.show()

print("Cancellation Rate by Month:")
display(cancellation_rate_by_month)
```



Cancellation Rate by Month:

	arrival_date_month	is_canceled
4	January	0.221275
3	February	0.232272
7	March	0.243675
0	April	0.304656
8	May	0.292280
6	June	0.303155
5	July	0.318019
1	August	0.321844
11	September	0.245441
10	October	0.236872
9	November	0.211011
2	December	0.268668

Next steps: [Generate code with cancellation_rate_by_month](#) [New interactive sheet](#)

- Business Impact:
- **Targeted Prevention Strategies:** Identifying months with higher cancellation rates allows hotels to implement targeted prevention strategies during those periods, such as special offers for non-refundable bookings or enhanced customer engagement.
 - **Dynamic Pricing Adjustments:** Knowledge of seasonal cancellation trends can inform dynamic pricing models, potentially adjusting rates to compensate for anticipated cancellations or to encourage bookings during low-cancellation months.
 - **Resource Allocation:** Helps in optimizing staffing and inventory during periods of high cancellation risk to minimize losses and maximize efficiency.
 - **Marketing Campaign Timing:** Guides the timing of marketing campaigns, focusing efforts on periods that traditionally have lower cancellation rates or offering more flexible options during high-risk months to manage expectations.